For any suggestions or feedback regarding these notes, please contact [Pragy Agarwal]

Pragy: DSA.. Hey Geetha, I've come up with a sorting algo that runs in $O(\log_2 n)$ time
Geetha: Bullshit – doesn't matter how smart you are, this is impossible!

For DSA problems, we have some "lower bounds" that tell us what is possible and what is impossible. This allows us to not waste our time on problems which are impossible.

CAP Theorem & PACELC theorem tell us what is possible what is not possible when we talk about distributed systems.

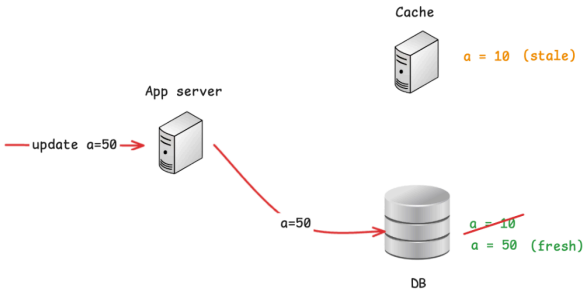
Consistency

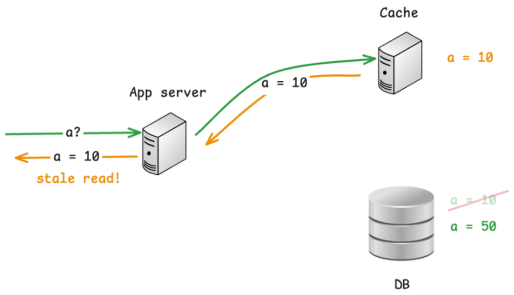
When can we talk about consistency?

1. Consistency is always about data
    - stateless app servers cannot be consistent or inconsistent — because they're stateless - they don't have any data
    - consistency is typically in context of database or caches
2. There have to be multiple copies of data (replication)
    - if there's only 1 copy of data - then it is guaranteed to be up to date (because reading & writing to the same place)
    - the only possible way for things to be out-of-date is when we've multiple copies of the same data.
    - for example
        - database + cache
        - or, having database replicas
3. There should be a read after an write/update
    - if the data is static/immutable, then there's no way of having stale reads - because the data is never updated
    - for example, after the contest is over, the leaderboard is always up-to-date (because no new submissions). So after the contest, we won't say that the leaderboard is consistent, we will say that consistency is no longer relevant.

Stale Reads

When old value of data is read after an update.
Stale reads can (not always) lead to bad user experience
If a stale read is possible, it means that we don't have consistency.





Immediate Consistency (aka. Consistent)

It is impossible for stale reads to happen - every read is guaranteed to read the latest data.
Only possible when all the "copies" are always kept in sync, all the time.
Immediate consistency is always good to have, but it is very hard to achieve (typically requires 2 phase commit - high latency, low availability)
Because it is hard to achieve, we do NOT always go with immediate consistency.

Eventual Consistency (aka. Not Consistent)

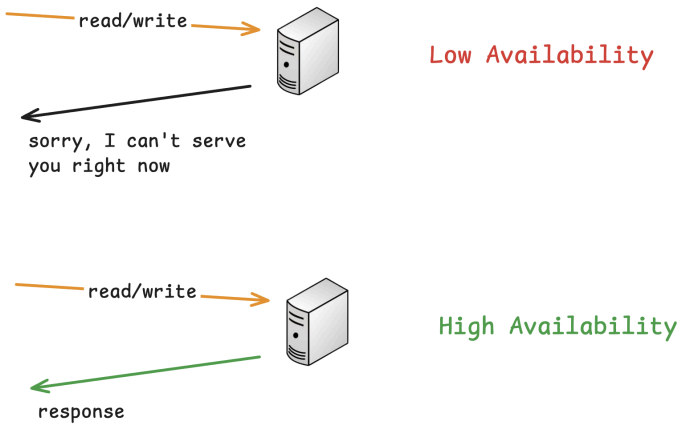
Is it possible for some reads to be stale for some time. But if we wait long enough, then eventually, we will be able to read the updated data.
Note: eventual does NOT mean that eventually the data will be latest (consistent) - it means that every write will eventually reflect in all "copies". It doesn't mean that data will stop being updated – it means that if you wait long enough, then any update made will be visible in the data.
In 99% of the cases, eventual consistency is good enough.

Data Loss — **No Consistency**

If it is possible for some data to be lost, then it means, that there are some updates that will never reflect in the data.
No matter how long we wait, the data will never become consistent. Data has been permanently lost.
It is very rare for systems to have no consistency (only relevant in case of analytics - when the trend is more important than individual data points)

Availability

Availability refers to whether a server can deny a request or not.
If a server can deny a request - then the system is "not available" or has "low availability".
If a server always satisfies the request - then the system is "available" or has "high availability".



Q: If a bad request comes to a server, and the server denies the request, is it low availability?

4xx error (bad request) or 3xx error (auth error)
In cases when the user is at fault, the server is allowed to deny the request.
No this is NOT low availability. We will still call the system as highly available.

Q: If a request comes to a server, but the server crashes (and doesn't respond), is it low availability?

Again, this will NOT be low availability. The client can just retry, and the LB will send the next request to a healthy server.

Low Availability

If a healthy request comes to a healthy server - but the server still refuses to provide the service (for some internal reason), then the service is said to have low availability.
Example

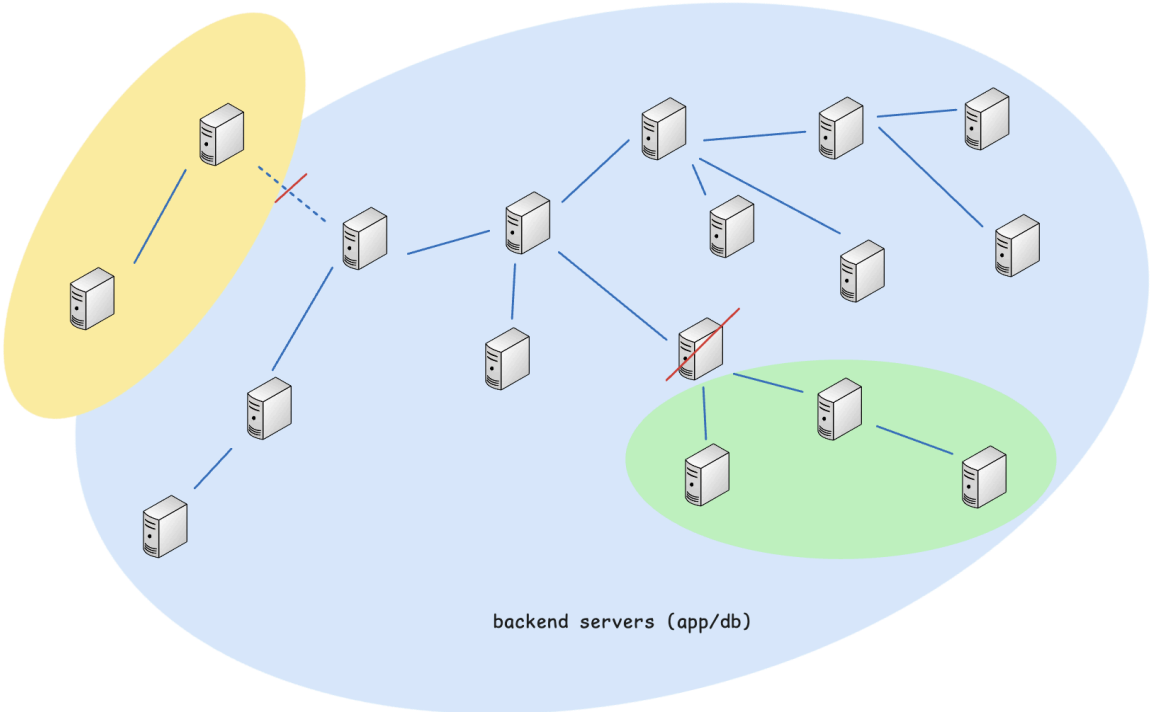
1. internal server error (bad code / unable to access certain critical resources)
2. some specific reason why the server wants to deny service
3. request times out (because the server is busy, not because the server is down)

Partition Tolerance

A system is ***partition tolerant*** if it can continue functioning seamlessly in the face of network partitions.

Network Partition

In a network of servers (distributed system), if some subset of the servers is not able to communicate with the other subset of servers, then we have a network partition.



Q: How likely is it for a network partition to happen?

In large scale systems distributed (google has 10 million+ servers) network partitions are not just likely, they **ALWAYS** exist.
Some subset is always disconnected - some node / network will always be down, leading to a small partition somewhere.
100% probability!

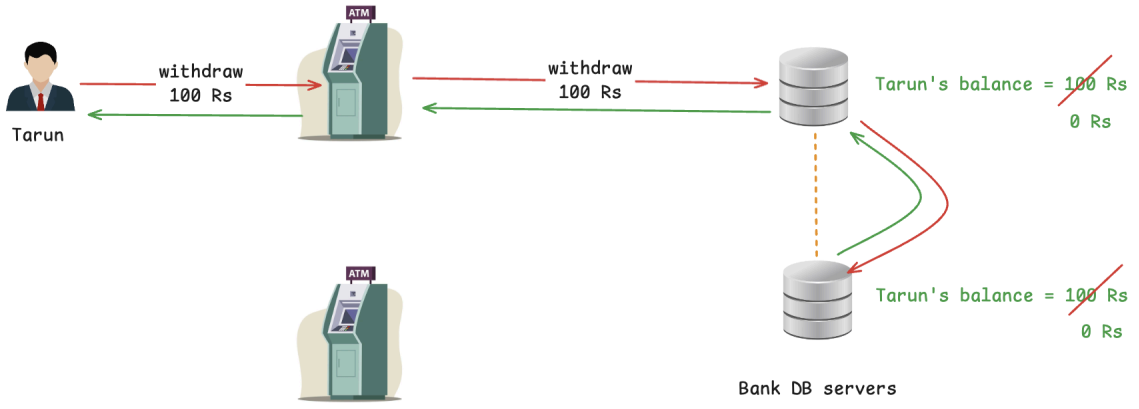
There's ALWAYS a partition somewhere.

Availability vs Partition Tolerance

Situation:

- Tarun wants to withdraw money from his bank account via the ATM
- In the backend, the bank will replicate the account information
    - because we don't want to lose the account data if some server/hard-disk crashes

When Tarun withdraw's money from ATM, the bank must update both copies of the data to reflect the new balance.



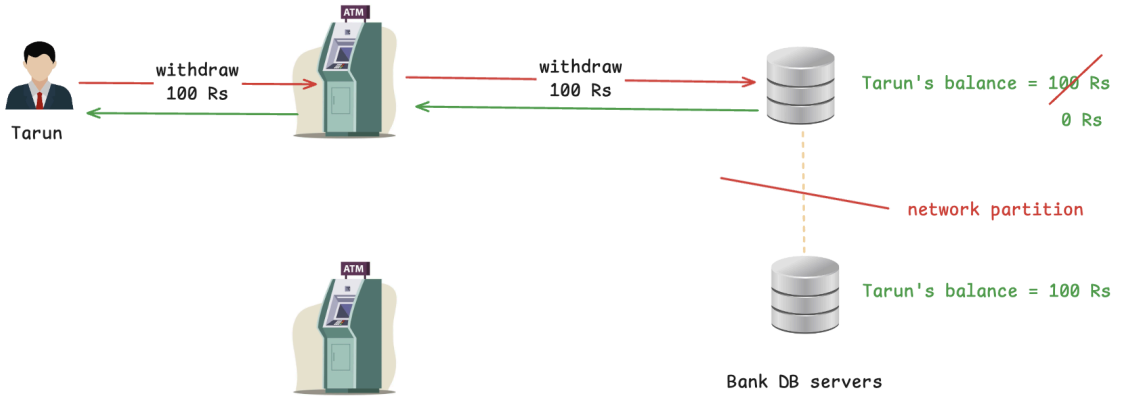
Now imagine that a n/w partition happens.
Once there's a n/w partition and the two bank database servers are not able to communicate, the following cases arise

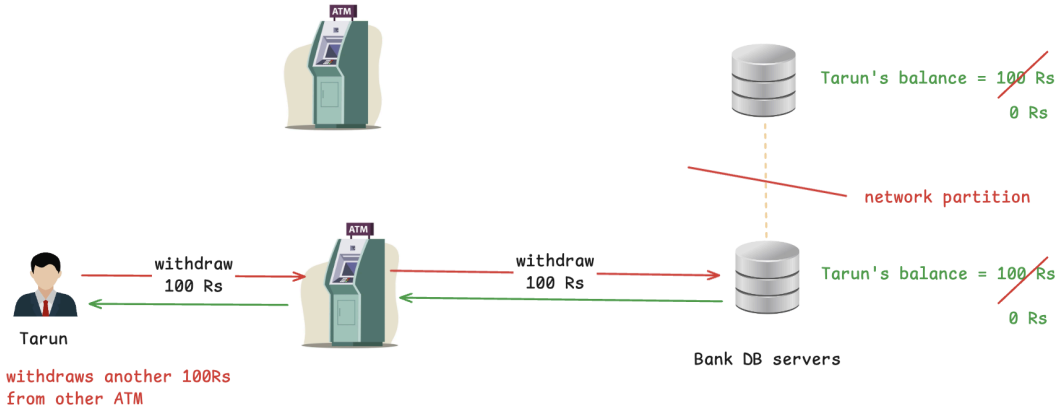
Highly Available system but not consistent (partition tolerant)

The bank returns success without updating both the servers - it updated the balance in just one server and gives the money to Tarun.

This will give us eventual consistency (stale reads are possible, but once the network is back up, then the db servers can sync up) and it will be highly available.

The data was inconsistent, but the system remained available in the face of n/w partition.

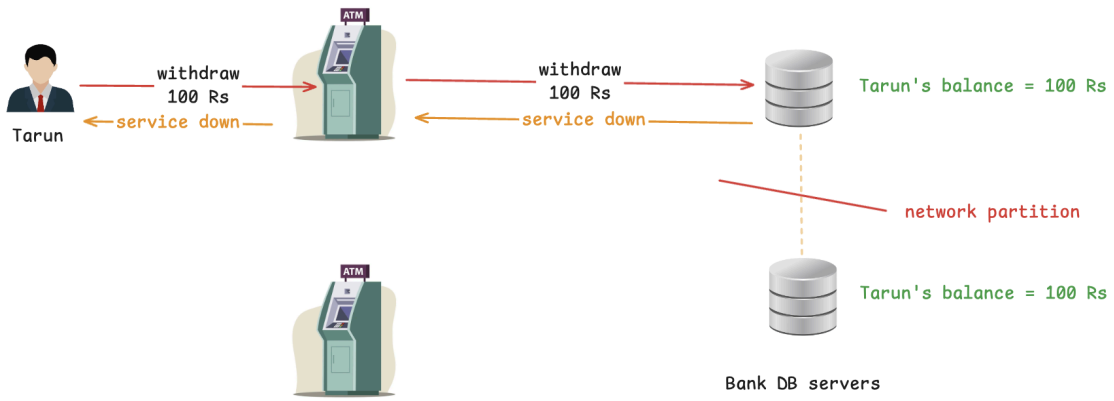




Consistent without Availability (partition tolerant)

The bank database can detect that it is not able to communicate with its replica, and it denies the service.
Low availability, because the request was healthy, the server was healthy, still, the request got denied!
But, we got immediate consistency — stale read was not possible.
However availability suffered — we had to deny the request.

Note: the redeem money functionality is down, but the check balance functionality is still up.
So the entire system is not down as a whole — the system is partition tolerant.



The system has continued functioning even in the face of n/w partitions.
There was low availability for the “money withdrawal”, but the system as a whole didn’t crash!

When will a system be not partition tolerant?

When any n/w partition brings down the entire system!

This is very very rare — most of the backend systems (99.99%) are designed to be partition tolerant.

In reality, the ATM system, and most banks follow A+P (eventual consistency)
reason: because the banks have found a way to "earn" from their own mistakes.

In reality, if we redeem more money than what you have due to a n/w partition, eventually the n/w partition will resolve and both the bank dbs will be able to communicate. Then they will do automatic "auditing" and they will realize that you redeemed twice.

Tarun's bank balance is now `-ve`. The bank will charge Tarun an overdraft fee + interest on the `100 Rs` he borrowed.

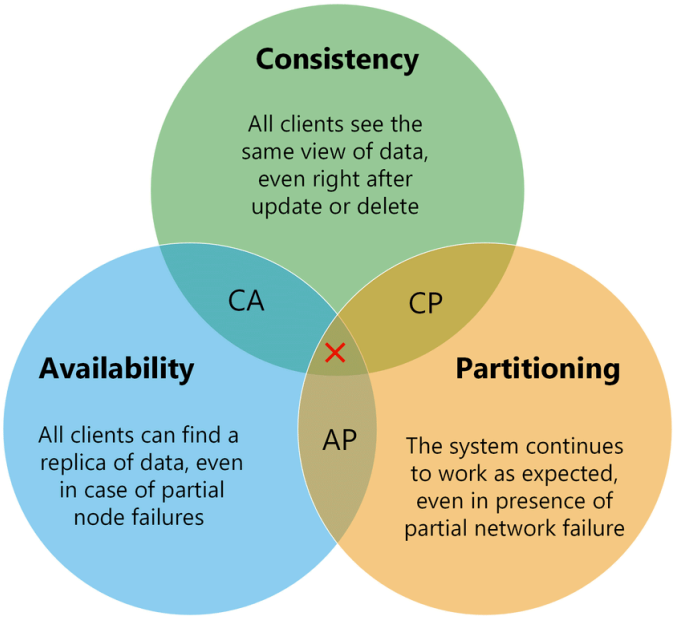
This is also why "refunds" take 5-7 business days, even though payment happens immediately. (note: typically the refund actually happens immediately - but in the worst case, it can take a long time - because eventual consistency)

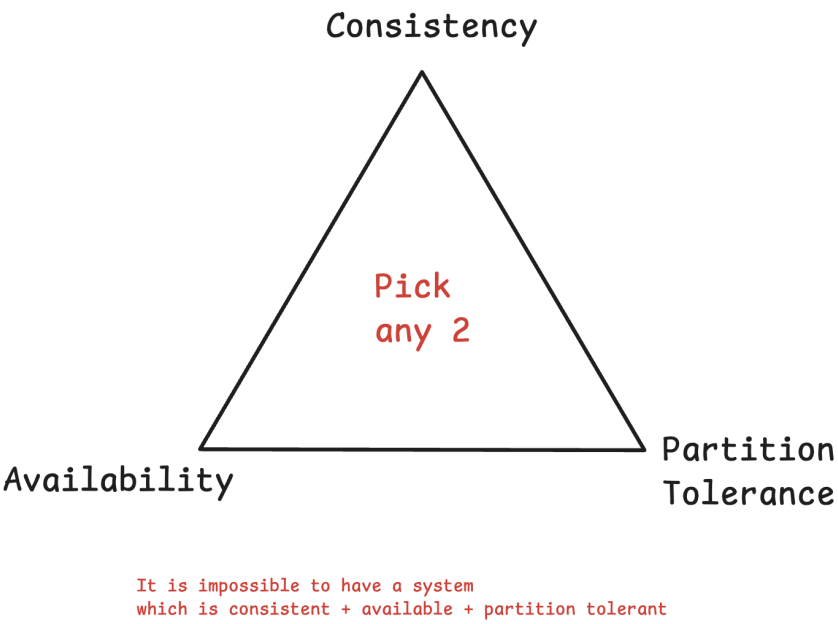
Theorems

Note: CAP/PACELC will apply only when you have replication (multiple copies of the same data, distributed across multiple servers, which are connected with a network)

CAP theorem

CAP theorem also gives you a fundamental limit of what you can achieve with distributed systems.





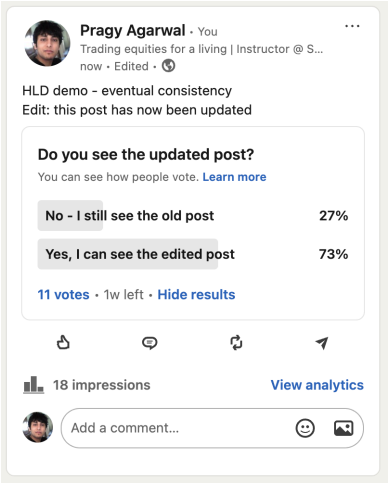
Google Spanner "seems" to violate the CAP theorem - they claim to provide all 3.
They don't actually violate CAP (they don't claim to violate CAP either)
They do some black magic with timestamps to practically achieve all 3

1. A+P Systems (high availability + eventual consistency)

Even if there's a partial failure (network partition), the system will still continue functioning (the users' requests will continue being served)
AP systems give up on consistency — they're eventually consistent

99% of all systems are AP. Because mostly, we do not care about immediate consistency

Social Media (twitter / facebook / instagram / ...)



if the user continues to see the old post, or if a new post doesn't appear in your friend's newsfeed immediately, it's not the end of the world — as long as the post gets updated eventually.

2. C+P Systems (immediate consistency + low availability)

If there's no network partition, only then, the system is able to serve the users.
The moment a network partition happens, the system may deny the users' requests (lose availability)

Rare. Only **0.9%** of the systems are C+P
When data is critical - example - financial transactions (mostly)
Banks, stock brokers (Zerodha)

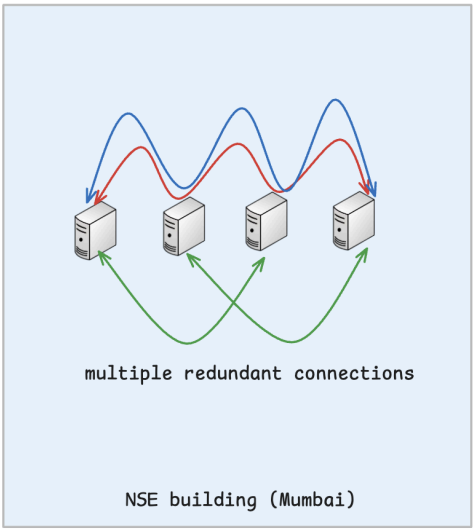
3. A+C Systems (consistent + available + but no partition tolerance)

Never denies the users' request, and always remains consistent as long as there's no network partition.
The moment a network partition happens, the entire system crashes as a whole!

Extremely rare. Only **0.1%** cases

Example, stock exchange (NSE, NYSE, SSE, BSE, …)
A+C systems are typically not geographically distributed — because we cannot afford network partitions.
In stock exchanges

- you have very powerful servers
- all the servers are located in the same building
- servers are connected with high speed fiber
- and multiple redundant connections



The chances of having a network partition in such a system are close to 0 (by design - extremely expensive to maintain)
Still, if a n/w partition happens, then the entire system has to shut down.

CAP theorem (another definition)

1st definition: pick any 2

2nd definition: when there's a n/w partition, you must choose between availability or consistency. You cannot have both.

As long as there's no n/w partition, you don't have to worry about anything. The moment a n/w partition happens, you can either choose Consistency, or you can choose Availability. You cannot get both.

Because in large systems
- there's always a n/w partition somewhere

Therefore
- you definitely want partition tolerance

So effectively,
- your choice is typically b/w A+P or C+P

PACELC theorem

PACELC theorem is an extension to the CAP theorem

- When there's a n/w **P**artition, choose b/w
  - **A**vailability, or
  - **C**onsistency
- **E**lse (even when there's no partition), there's trade off b/w

- **L**atency, and
- **C**onsistency

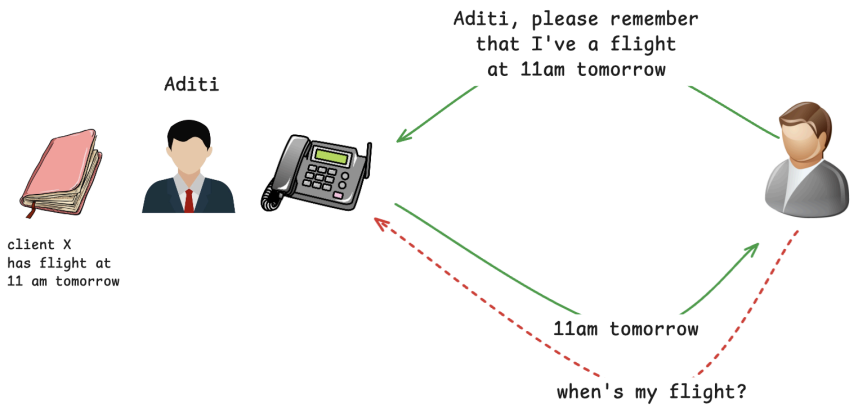
Just like in DSA, we have a tradeoff b/w Time Complexity and Space Complexity, we also have a tradeoff in HLD b/w Consistency and Availability, and Consistency and Latency.

Basically

- if you choose consistency, then you have to give up on Availability and Latency.
- if you want your system to always be consistent, then
  - it will be slow (high latency), and
  - it might deny requests (low availability)

Proof by Story

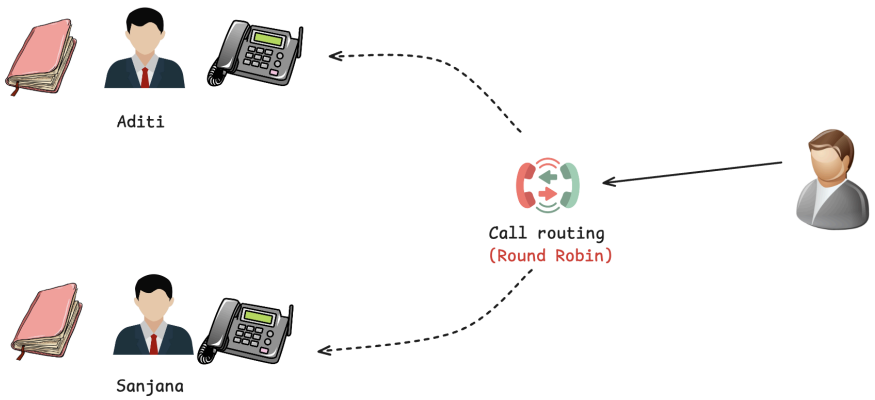
Aditi's Reminder Service



Vertical Scaling (Scaling Up)**:** In the start, Aditi was handing this service along with her day job.. but as the number of requests increases, she quit her job and started doing this service full time.

Horizontal Scaling (Scaling Out)

Because Aditi was not able to handle all the calls herself, she hired Sanjana to help her out.



Q: Should we maintain one shared diary, or two independent diaries?

If we go with a single diary, then

1. Aditi & Sanjana will have to be in the same room (colocated geographically - can't be distributed far apart)
2. Both of them will fight for the diary - the diary will become a bottleneck

It is better to have separate diaries

1. they can be located far apart (Sanjana can work from home)
2. the diary is not the bottleneck.

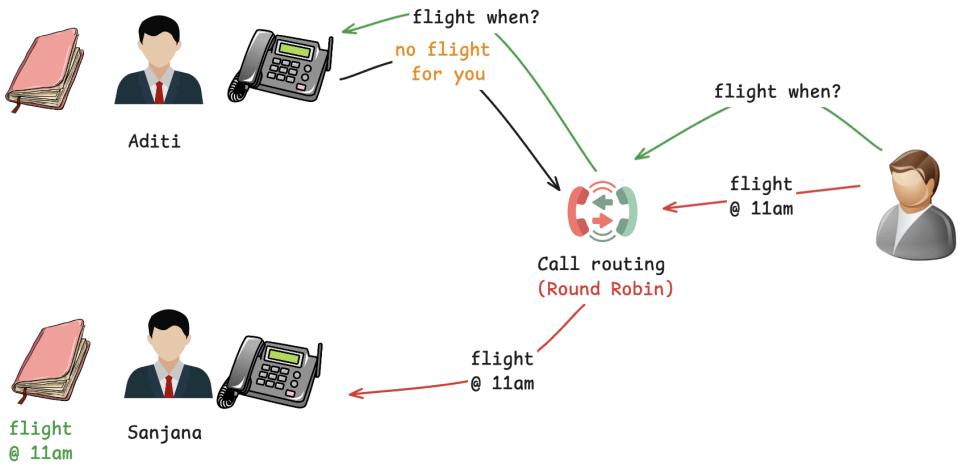
So we go with the 2 diary approach (because a single diary would render our scaling useless)

Angry Customer

Customer: *"Yesterday I called you to remember my flight at 11am in the morning. But then I called you at 8am to ask for my flight timing, and you said that you don't remember. You lost my data, and I missed my flight."*

Q: Why did this happen?

When the write request came, it went to Sanjana, but the read request came to Aditi, and Aditi's data was not in sync with Sanjana's data.
Consistency Issue



Protocol 1 - Sync eventually (at night)

They decide to copy each other's data at each night before they close the shop.
So that in the morning, both have the same copy of data.

Q: Can there be stale reads still?

Yes, until the sync happens (at night) any reads before the sync can potentially be stale.
But if we wait long enough, eventually, the data will be in sync.

So, the data can be stale (low consistency)
but, when a write request comes, we only have to write at 1 place (low latency)

Low Consistency + Low Latency

Protocol 2 - Sync immediately

They decide that whenever one of them gets a call

1. write the data in their own diary
2. call the other person
    - get the other person to write the data as well in their diary

3. only when the other person responds with a success, we can return success
    - if the other person returns error, then we should delete from our diary (rollback) and return error

This protocol will give them immediate consistency - the data will always be in sync *(because I'm always writing to both diaries)*
Note that this will lead to higher latency, as when Aditi is calling Sanajana to copy to write, the client is waiting on the phone.
Also, the throughput will suffer — because now, every call is being multiple twice – it is going both to Aditi and Sanjana. This means that the horizontal scaling that we did is now suffering

High Consistency + High Latency

Q: How does Google doc handle this situation?

Gives you eventual consistency (you're allowed to make changes even when your n/w is down — the changes will sync up eventually, once your n/w is restored)

How? By performing conflict resolution later.

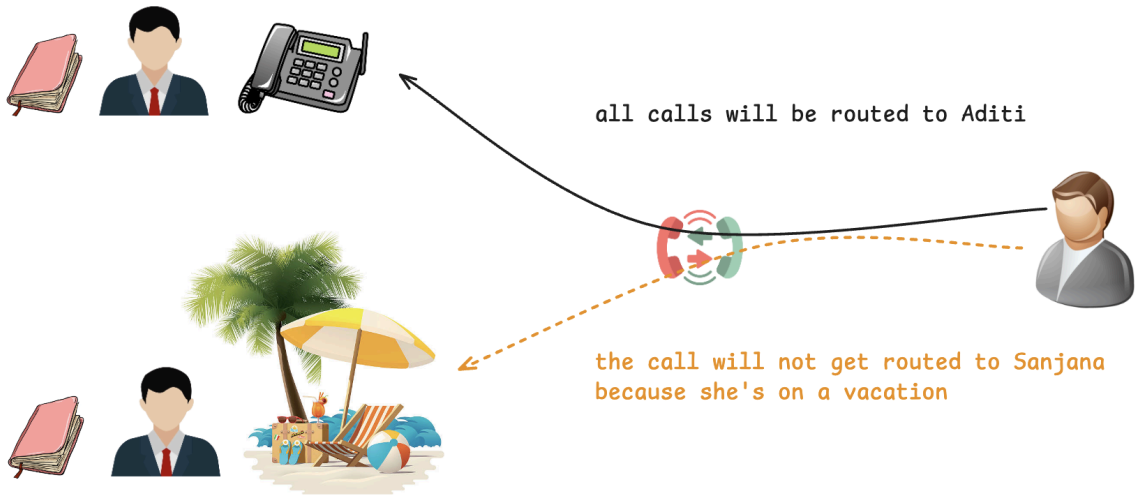
[Conflict-Free Replicated Data Types (CRDTs)]

CRDTs are very powerful, but also very complex. Google Docs doesn't actually use CRDTs, but it uses a custom protocol that is a little weaker (but similar) to CRDTs.

Vacation Time

Sanjana decides to go on a vacation.
No availability issue - Sanjana is just on a vacation.
No n/w partition — Sanaja is just on a leave. The calls can be routed to Aditi.

When Sanjana comes back, before she starts taking any calls, she can just sync up with Aditi's diary - **no consistency issue here**.



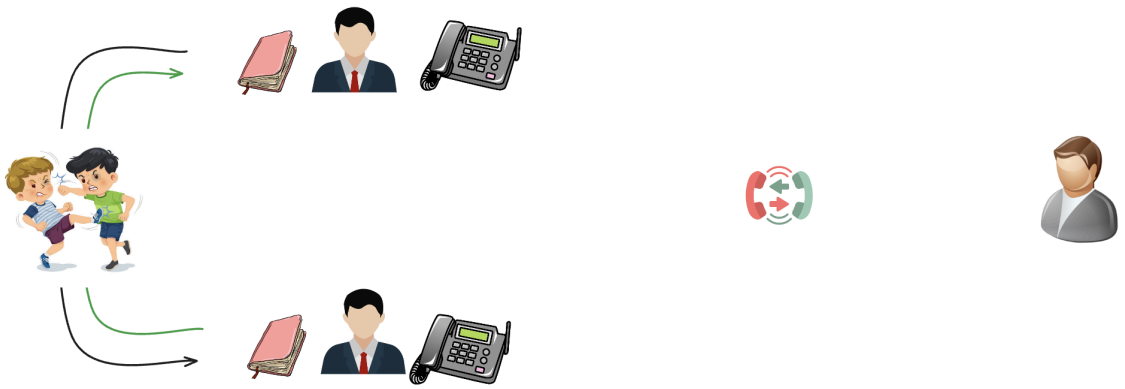
Protocol 1 - Sync eventually (at night)
+ Protocol 2 - Sync immediately

in both the protocols, because Sanaja is simply not present, Aditi can just keep writing in her own diary (even in protocol 2).
When Sanjana comes back, she can sync up before serving any calls.

So no availability issue, no consistency issue. Why? Because there's no n/w partition. It's just that some server (Sanjana) is down.

Big Fight

Sanjana & Aditi have a big fight, and they refuse to talk to each other **(n/w partition)**



Protocol 1 - Sync eventually (at night)

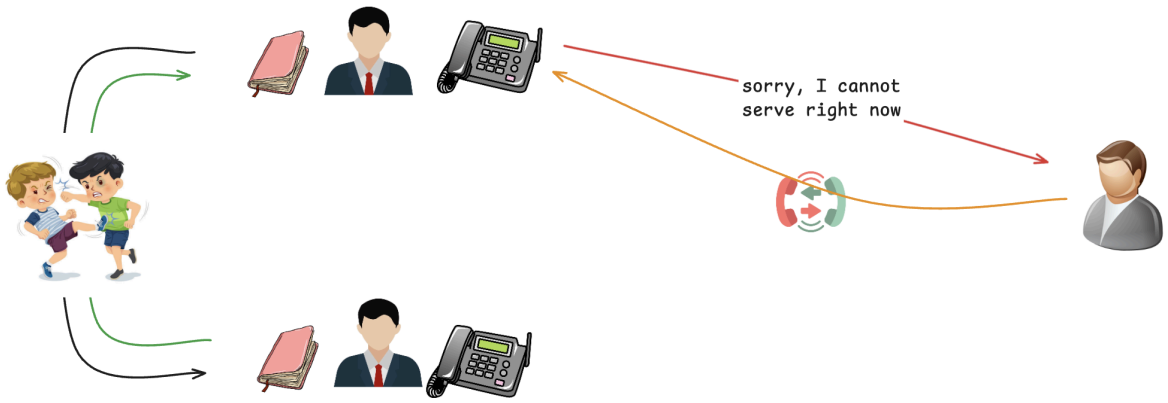
No worries — both of them can continue taking calls & continue writing in their own diaries. Eventually, the fight will resolve (n/w will be fixed) — they can just sync up their diaries when they're friends again.

Data will remain stale until the fight resolves. But the service will continue to be available.

Partition ⇒ High Availability + Eventual Consistency

Protocol 2 - Sync immediately

The protocol mandates that Aditi must write to both the places, but she's not talking to Sanjana
So when she gets a "write" call, she must deny.
Note that she can still continue serving "read" requests from her own diary! But any write requests must be denied until the fight resolves.



Partition ⇒ Immediate Consistency + Low Availability

Note that only the write requests are unavailable – the read requests continue. ***But overall, we will say that the system has low availability.***

8.43 ⇒ 8.55

Replication

Replication: Explicitly (by choice) having multiple copies of the same data.

Isn't replication bad?

RDBMS - we study normalization.
While studying normalization, we say that redundancy (duplicate data) is bad, because it leads to various anomalies (read / write / delete / ...)

Course	Student

HLD	Tarun
HLD	Sanjana
LLD	Harini
DBMS	Sanjana

If we update the name from HLD ⇒ High Level Design
But, we fail to update all places (inconsistency)

Course	Student
High Level Design	Tarun
HLD	Sanjana
LLD	Harini
DBMS	Sanjana

If we delete Sanjana
we have mistakenly deleted the course DBMS (because Sanjana was the only entry with DBMS)

Course	Student
HLD	Tarun
~~HLD~~	~~Sanjana~~
LLD	Harini
~~DBMS~~	~~Sanjana~~

Normalized design (which gets rid of these anomalies)

Course ID	Course Name
1	HLD
2	LLD
3	DBMS

Student ID	Student Name
1	Sanjana
2	Tarun
3	Harini

Student ID	Course ID
1	1
1	3
2	1

3	2

Why do we need replication?

Replication is the ONLY way that we can protect ourselves from data loss.

"D" in ACID stands for durability - any committed transaction will be persisted on the disk. So if there's a power failure, once the system restarts, we will find that transaction in the database, it will not be lost.

ACID Durability is about persistent vs volatile memory. ***It does NOT protect you from HDD failures****.*

" The only way to prevent data loss in case of HDD failures is replication *(have multiple copies of the same data across multiple disks).* "

Replication vs Sharding

There's no point in replicating the data within the same server (useless redundancy) — we're just wasting disk space, and if the server crashes, we still lose data.

You always replicate across multiple servers (useful redundancy)

	Replication	Sharding
	Having multiple copies of the same data across multiple servers	Partition data horizontally across servers
	Replication == Copying	**Sharding == Splitting**
	Different servers (replicas) have the same data	Different servers (shards) have different data
Pros	1. Minimizes the chances of data loss 2. (only) Read throughput is improves, because you can read from any copy (the read load is distributed equally across all the replicas)	1. Allows us to store more data than a single server's capacity (scaling out) 2. Both read & write throughput increases (the read & write load is distributed equally across all the shards)
Cons	1. It leads to increase in data size (so capacity is decreased - if you have 3x replication - each piece of data is copies in 3 places, then your overall data storage capacity has decreased by 3x)	1. Complex 2. Requires consistent hashing for routing

2. It worsens write throughput
   (because you now have to copy the data to multiple places - which means extra writes)

Q: Which one should we use?

Both! Together.

You have more data than a single server can handle, and you also don't want to lose the data.

Large systems have both sharding & replication at the same time.

How does sharding happen with replication? We will see this in Lecture 8

Q: What about Caching?

Caching is a special form of replication.
In caching, your data is in 2 places
1. database, and
2. cache

It's not pure replication — caching will not save you from data loss - because caches are volatile (they store data in RAM typically).

Cache
1. doesn't save you from data loss (because cache is volatile)
2. improves read throughput (good)
3. worsens write latency & throughput (bad)

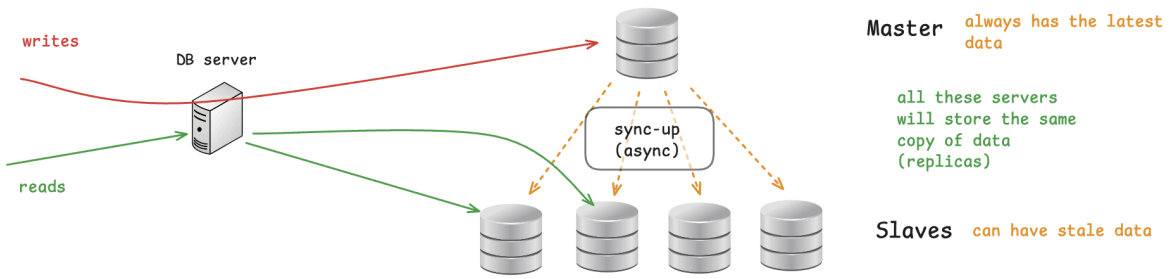
Master-Slave Replication

Terms "Master" and "Slave" have a lot of historical baggage - and some communities don't like these words.

a.k.a.

- Leader - Follower
- Active - Passive
- Primary - Secondary

Basic Setup



Typically in Master-Slave replication, we have an odd number of total servers (replicas).

Replication Factor (X): Number of total copies of the same data
`(X = 1 master + slaves)`

Note: replication factor is the number of copies. It is not the total number of servers. If the database is sharded, we will have shards & replicas. A database can have `S=100` shards, each shard having `X=3` copies each, for a total of `N=300` servers.
Typical values of `X = {3, 5}`

Mandatory reading (SDE2 or higher)

Master Slave: MySQL

What happens if the master goes down?

A re-election will happen. One of the slaves will get elected as the new master.

What happens when the old master comes back online?

It will join as a new slave.

How does master election happen?

Typically happens via some "distributed configuration management system" (ex. Zookeeper)
But there are other algos as well.
[We will deep dive on this in the Zookeeper class]

How do the slaves sync with the master?

Asynchronously, using [Write Ahead Log]

Where do the writes happen?

Always happen at least in the master.
(in some configurations, the writes can also happen in a subset of the slaves)

Where do the reads happen?

In a subset of the slaves

How many slaves?

Typically, an even number (so that the total number of servers is odd, because there's 1 master)

Why Odd number of servers?

So that we can have a clear majority - easy to do tie breaking (in quorum)

Master Slave with No Consistency

	Where?	Availability	Latency
Reads	at any 1 random slave (random slave is diff for each read)	Very High	Very Low
Writes	only at the master	Very High (if master goes down, it will just be re-elected)	Very Low

Q: Can there be data loss?

Yes.
Writes are happening only at the master. Slaves sync up with master periodically.
If the master goes down (HDD failure) with unsynced changes => these changes are lost forever!

Q: Can stale reads happen?

There can be stale reads because the slaves sync up only periodically.
Note: data loss is even worse than stale reads.
Because this config gives us data loss, we don't even care about stale reads.

Master Slave with Eventual Consistency

we will use this when eventual consistency is good enough

	Where?	Availability	Latency
Reads	at any 1 random slave (random slave is diff for each read)	Very High	Very Low
Writes	Master + 1 random slave (random slave is diff for each write)	Decent (if master goes down, it will just be re-elected. If a slave goes down, just choose another random slave)	Decent

Q: Can there be data loss?

No. Because the data is in at least 2 places.
What happens if the HDD fails in both the places? That will certainly lead to data loss.
What happens if Thanos attacks earth? Supernova? Black hole? ... if earth explodes, that means data loss.
Don't worry about it. It's very rare.
If we still wish to minimize your data losses, then you just write to more places.

Q: Can stale reads happen?

Yes. The reads are happening at any 1 random slave, and the slaves are not guaranteed to always have the latest data.

Master Slave with Immediate Consistency - Write Everywhere

we will use this when we require immediate consistency, and the writes are rare (read heavy system)

	Where?	Availability	Latency
Reads	at any 1 random slave (random slave is diff for each read)	Very High	Very Low
Writes	*all nodes* master + all slaves	Low (if even 1 server goes down, your writes have to be denied)	High (you've to wait for all servers to respond - limited by the slowest server)

In write-everywhere the master's boundary is blurred.

Q: Can there be data loss?

No. All the copies are always in sync, and each write is replicated in multiple places.

Q: Can stale reads happen?

No. All copies are always in sync

Master Slave with Immediate Consistency - Quorum

we will use this when we require immediate consistency, and the writes are frequent (write heavy system)

Idea: Improve the writes at the cost of reads. In write everywhere, the reads are very fast, but the writes are very slow. If we trade off, then both will be decent.

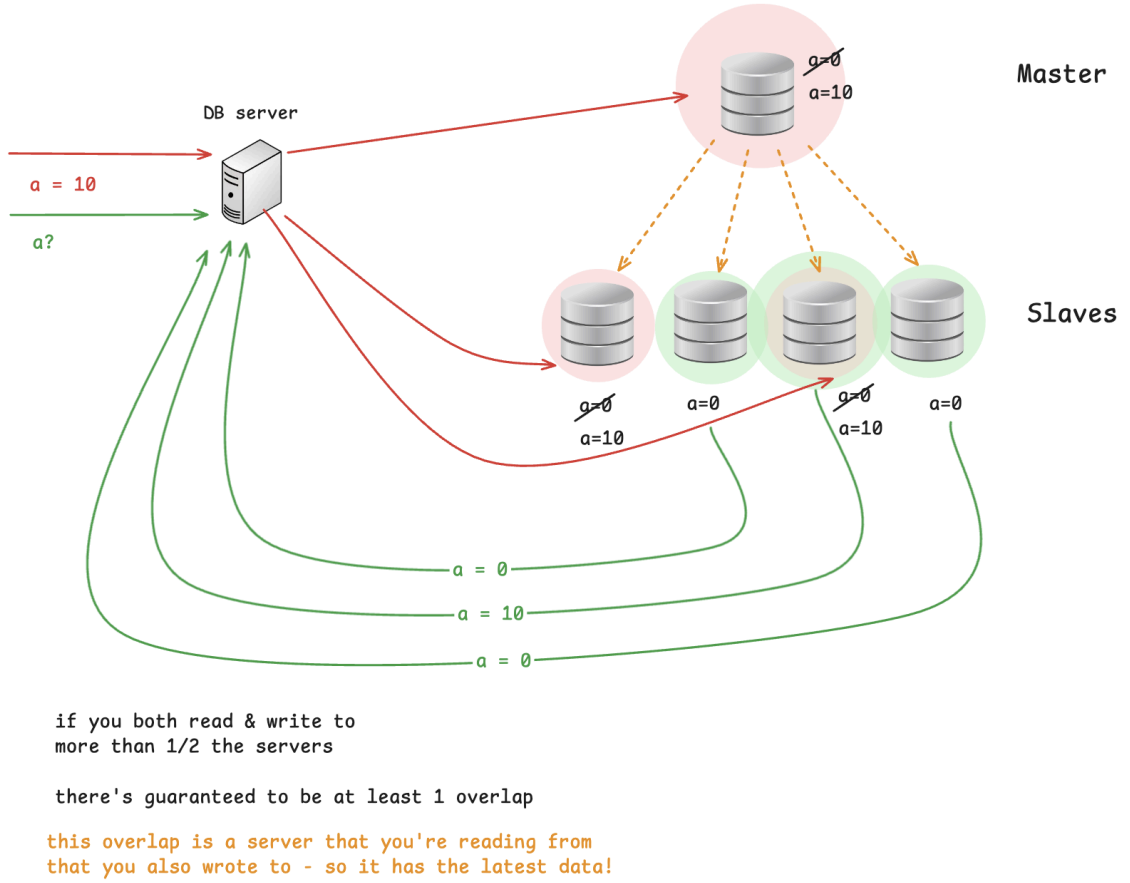
	Where?	Availability	Latency
Reads	to a majority (master + at least a random half of the slaves)	Decent (at least half the servers have to go down for the request to be denied)	Decent
Writes	to a majority (master + at least a random half of the slaves)	Decent (at least half the servers have to go down for the request to be denied)	Decent

Q: Can there be data loss?

No

Q: Can stale reads happen?

No.
You always read the latest data, because there's at least 1 overlap (there's at least one server that you're reading from that also got the latest write)
Since different reads will return different values, we will always maintain the timestamp whenever writes happen.
And then when reading, we will return the value which has the latest timestamp.



Q: What if some servers are down? How does Quorum consider the number of servers then?

…

Tunable Consistency - Unlocking the Spectrum **Wait, there's more!?**

X = Replication factor (number of copies of the same data)
R = minimum number of servers which must respond with data for the read to succeed
W = minimum number of servers which must acknowledge the write for the write to succeed

R = 1, W = 1

same as Master-Slave with no consistency
write at master, read from any 1 random slave

R = 1, W = 2

same as Master-Slave with eventual consistency
write at master + any 1 random slave, read from any 1 random slave

R+W <= X

we will get eventual consistency

R+W > X

(the number of servers that we're writing to + the number of servers that we're reading from) is more than the total number of available servers.
=> there's at least one server that is being both written to and being read from (there's an overlap)!
=> we've immediate consistency!

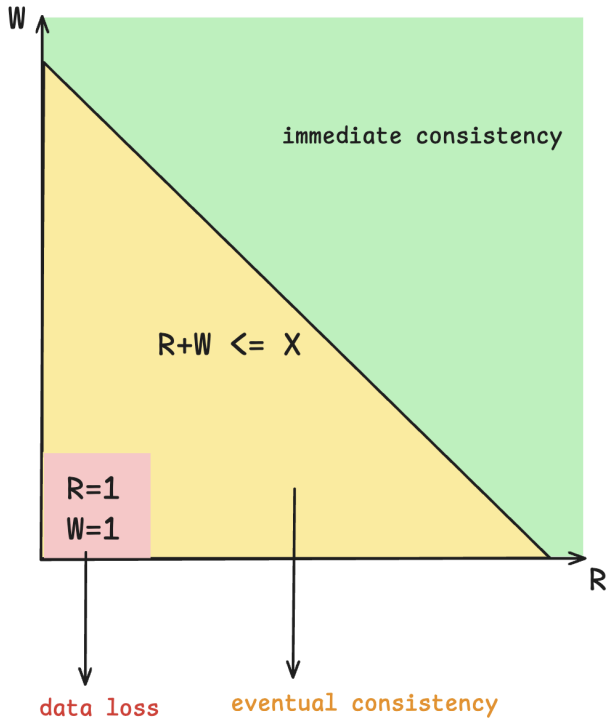
`R = 1, W = X` **=> write everywhere**
`R = W = (X+1/2)` **=> quorum**

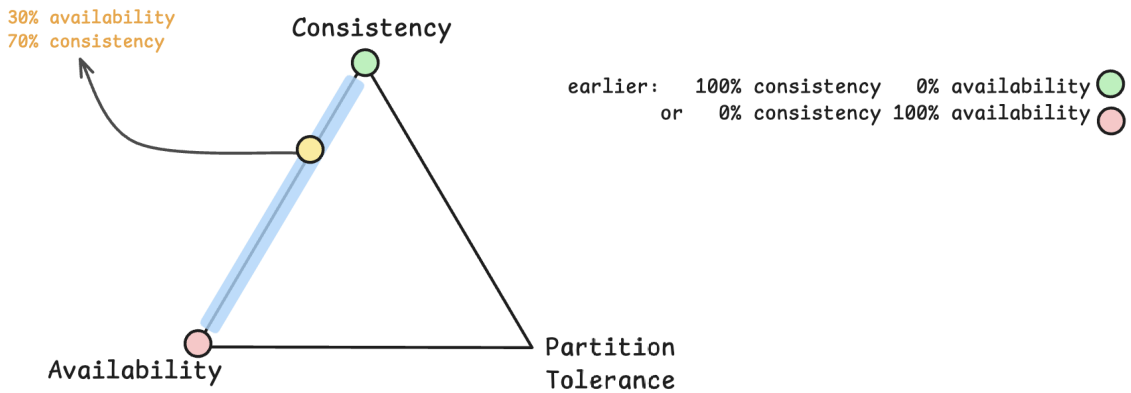
Since we can now choose any values of R+W, this gives us a tremendous amount of power.
If R is low => reads are faster & more available (because you're reading from only few servers)
If W is low => writes are faster & more available
If R+W > X => system is consistent

As R+W increases,

- **availability becomes lower** (because you need more servers to be alive for the reads/writes to succeed)
- **consistency becomes higher** (because you're more likely to find an overlap - a server which you're reading from which also has the latest write)

We can not only decide our consistency, we can also **tune the system for read-heavy or write-heavy operation**!
In fact, we no longer are limited only to availability or consistency, we can choose any point on the spectrum!





How to write to more than one place atomically?

Two Phase Commit (2PC)

1. **Theory:** https://www.youtube.com/watch?v=-_rdWB9hN1c
2. **Explanation 1:** https://www.youtube.com/watch?v=eltn4x788UM
3. **Explanation 2:** https://www.youtube.com/watch?v=7FgU1D4EnpQ
4. **Implementation:** https://www.youtube.com/watch?v=oMhESvU87jM